

MEMORIA TÉCNICA DEL PROYECTO:

GESTIÓN LIGA 1GSY

Autor: Jose Antonio Rodríguez Vera

Curso: 1º Desarrollo de Aplicaciones Web (DAW)



- 1. Introducción y Objetivo del Proyecto

Este documento es la memoria técnica de la aplicación para gestionar la liga de e-sports de la empresa 1GSY, arrays, matrices, colecciones dinámicas y control de excepciones.

El programa hace comprobaciones constantes para que la liga tenga sentido (por ejemplo, que no jueguen sancionados, que no se repitan IDs o que los equipos tengan exactamente 5 titulares).

- 2. Estructura de Clases y Jerarquía

Para no tener todo el código amontonado en el Main, he diseñado una estructura de clases basada en la herencia y el polimorfismo.

- 2.1. Clase Abstracta PersonaLiga

Tal y como pedía el requisito 5.1, he creado la clase abstracta PersonaLiga. Esta clase sirve como "molde" base. Tiene los atributos de identificador, nombre, nickname, edad y salario base. He hecho que el método calcularCosteMensual() sea abstracto para obligar a que las clases hijas lo programen a su manera.

- 2.2. Jugador y Entrenador (Herencia)

De la clase base nacen Jugador y Entrenador. El jugador añade cosas como el rol (TOP, MID, etc.), los niveles mecánicos y si está sancionado. El entrenador maneja su experiencia y sus victorias. Además, he implementado la interfaz Entrenable en la clase Jugador, añadiendo los métodos entrenar() y calcularRendimiento().

- 3. Gestión de la Plantilla: Arrays vs Listas

He usado arrays fijos y listas dinámicas con un sentido real. Así es como lo he aplicado en la clase Equipo:

- 3.1. Array Fijo para Titulares

He usado `private Jugador[] titulares = new Jugador[5]`.

He elegido un array fijo porque las reglas del juego son estrictas: un equipo solo puede jugar con 5 personas. No tiene sentido usar una lista infinita para algo que tiene un límite obligatorio. Si se intenta meter un sexto titular, el programa da un error (Excepcion).

- 3.2. ArrayList para Suplentes

Para los suplentes he usado `private ArrayList<Jugador> suplentes = new ArrayList<>()`;

El banquillo es flexible; un equipo puede tener fichados a 2 suplentes o a 10. El ArrayList es perfecto aquí porque crece y encoge automáticamente al fichar o eliminar jugadores.

- 4. Estructuras Dinámicas de Control (Punto 5.7)

Para que la aplicación no falle ni tenga datos corruptos, he usado la estructura HashSet. Esta colección no permite duplicados.

```
private static HashSet<String> idsRegistrados = new HashSet<>();
```

```
private static HashSet<String> idsPartidos = new HashSet<>();
```

Cada vez que intento crear un Jugador, Entrenador o Partido, el programa consulta el HashSet. Si el ID ya existe, bloquea la acción. Es la forma más eficiente y rápida (complejidad $O(1)$) de garantizar que no hay dos DNI o dos códigos de partido iguales en todo el sistema.

- 5. Calendario y Matriz Bidimensional (Punto 5.8)

Para guardar los puntos de la liga he usado una matriz bidimensional: `private static int[][] matrizPuntos = new int[10][10]`;

Las filas representan a los equipos y las columnas representan las jornadas. Al jugar un partido, el programa busca el índice del equipo local y el índice del equipo

visitante, y les suma los puntos en la columna de la jornadaActiva. Esto me permite luego imprimir la clasificación y saber exactamente cuántos puntos sacó un equipo en la jornada 2 o en la 3.

- 6. Cola de Partidos (FIFO) y Pila de Historial (LIFO)

El proyecto demuestra cómo funcionan las estructuras FIFO y LIFO en un entorno real.

- 6.1. Cola de Partidos Pendientes (LinkedList)

He usado LinkedList para almacenar los partidos creados mediante el método offer() y remove(). Al programar partidos, se ponen en la cola. Como es FIFO (First In, First Out), el primer partido que se anota es el primero que sale para jugarse. No te puedes saltar el orden cronológico.

- 6.2. Pila del Historial (ArrayList simulando LIFO)

Para guardar cada movimiento (fichajes, sanciones, creación de equipos), he usado una lista que recorro y leo al revés. El último elemento en entrar es el primero en salir (LIFO). Esto me ha permitido crear el método deshacerAccion(), que coge el último registro y lo borra.

- 7. Gestión de Excepciones (Punto 6)

He creado mis propias clases de Excepción:

- RolNoDisponibleException: Salta si intentas meter de titular a un jugador cuyo rol (ej. JUNGLE) ya está cogido por otro titular del equipo.
- JugadorSancionadoException: Salta justo antes de jugar un partido si el programa detecta que uno de los 5 titulares tiene el booleano sancionado = true. Bloquea el partido al instante.
- EquipoNoEncontradoException: Para gestionar las búsquedas manuales del usuario.
- PokemonNoReconocidoException: Verifica que el Pokémon elegido pertenezca al Enum oficial de la liga.

- 8. Partidos e Incidencias

Para registrar lo que ocurre, creé las clases Partido e Incidencia.

Cuando ocurre algo anómalo (una expulsión o un error técnico), se crea un objeto Incidencia y se guarda en un ArrayList general. Esto permite buscar luego las faltas filtrando por nombre de equipo o por nickname de jugador. Si la incidencia es una "SANCION", el sistema actualiza automáticamente el estado del jugador para que no pueda jugar el siguiente encuentro de la cola FIFO.

- 9. Conclusión Final

El código ha sido un reto de organización. El menú (que tiene 22 opciones) hace comprobaciones. Antes de jugar, compruebo que haya cola; antes de sacar la clasificación, compruebo que haya equipos; antes de pasar de jornada, vacío la lista de partidos jugados de la jornada anterior.

Con este diseño, la aplicación no es solo un almacén de variables, sino un sistema real que controla las reglas de los e-sports de principio a fin, aplicando encapsulamiento.